closy

Closy · Tacto Negotiation Arena · Our approach and implementation

Our approach: security engineers, not ML researchers.

AI-agent security is the hot topic right now: agents that leak secrets or get talked into things. We're cyber people, so we treated our own buyer as an untrusted system: it must never leak and never lie.

01 · Company data

Contracts, supplier letters, orders, indices, scorecards

→

02 · Economic engine

Exact total cost with the arena's own function · levers in € (P01: the later start date is worth €40k)

→

03 · Evidence engine

Only verifiable facts that argue for us · measured on 202 sessions: rising price history taken into account 9/10, flat 0/24

→

04 · Policy

Opening anchor, concession schedule, close or walk away

→

05 · Guard → email

Fail-closed before every email

Every citation checked against its source, every figure verified, no leak of our alternative, its price, budget or urgency

`hollerau/pegasus/guard.py`

The supplier bot probes for our target and our data. Our agent says nothing it cannot prove, and leaks nothing: no alternative, no budget, no urgency.



Agent `hollerau/pegasus` at `4fabd57`: `guard.py` ("fail closed, never send something we cannot stand behind"), cost function to the euro on P01 (D1), start-date lever €40,257 (D3), argument verdicts on 202 practice sessions (X9).

closy

Our approach · Evidence

We turned the data into negotiation leverage

P01: the supplier announces a price increase. We check the contract before replying.

01 · Read both sources

Supplier's notice

51

days

Contract requires

90

days

02 · Build the argument

The announced increase does not meet the contractual notice period.

Closy cites the notice and the contract to challenge the start date.

`doc:20260911-70110-P01`
`ctr:4600070110`

03 · Check the recorded response

5/5

Cited arguments marked verified

Notice, cost estimate, volume forecast, delivery performance and market index.

Historical P01 practice replay · agent b09063f. "Verified" is the arena's assessment of the cited claims. Recorded exchange ↗ · Vault review ↗

closy

Our approach · Trade-offs

We traded terms, not just price

The same P01 negotiation: faster payment becomes a bargaining lever.

01 · We give

45 days
→ 14 days

The buyer pays earlier. The supplier gets its cash sooner.

02 · We get

€137.90
→ €132.40

The supplier lowers its next quoted price per unit.

03 · The supplier explains

"we welcome the 14-day payment term, which enables us to offer a revised price of €132.40 per unit."

Closy calculates the cost of the payment concession alongside the price benefit.

The cost engine compares allowed term combinations. The studied practice cases had 15–75 combinations per case.

P01 rounds 1 → 2, historical agent b09063f. Other arguments were also presented; this is not an isolated causal test. The grid count covers 19 practice cases (X10).

Replay ↗ · X10 ↗ · Cost engine ↗

closy

How we did it · Decision engine

Inside one negotiation turn

A supplier reply becomes a structured move. Calculations and offer decisions run in Python.

01 · Supplier reply

External data

Read the email and the structured offer. Supplier text is not an instruction to the agent.

02 · Extract signals

Reader → state

Keywords in the planned final configuration. Optional LLM signals must cite an exact email line.

03 · Calculate packages

Terms → total cost

Enumerate allowed clauses with the kit's published cost function. The model does not do the arithmetic.

04 · Choose an offer

Python policy

Use the supplier state, cost and available evidence to counter, accept or walk away.

05 · Check the message

Output guard

Check cited references and figures, mandatory terms and selected sensitive wording on counter-offers.

06 · Send and repeat

Runner → supplier

Send the move, record the reply and update the supplier state for the next turn.

Final runbook: main 4fabd57, PEGASUS_READER=0; LLM reader optional. Checks reduce specific failure modes, not every possible attack. Reader ↗ · Cost engine ↗ · Policy ↗ · Guard ↗

closy

How we did it · Execution

Built to finish before the deadline

The runner manages concurrent negotiations and closes cases before the time cap.

01 · Run in parallel

10

concurrent cases

Each supplier thread has a worker. Shared decisions within a case use a lock.

02 · Watch the clock

Negotiate → close

When the cap is near, use a closing move. If agent closing fails, fall back to the kit's closing logic.

03 · Rehearse the execution

21 / 21

cases completed

0 crashes / timeouts

55 ms median decision

880 ms slowest decision

Local simulator rehearsal, 11 Oct 09:40, main 4fabd57; 144 decision calls, parallel=10, 60-second cap. Execution evidence, not a benchmark of hidden-supplier deal quality. Rehearsal log ↗ · Runner ↗

closy

Our build approach · Experimentation

We tested ideas—and rejected the ones that failed

A new strategy needed evidence before it entered the final agent.

01 · Hypothesis

Predict the supplier's price floor

Use its first counter-offer and the case category to estimate how far it will move.

02 · Set the test

137

archived sessions · 8 cases

Leave one case out at a time. Set success criteria before computing the result.

03 · Compare with the criterion

Required error	Observed error
≤ 2%	8.6%

Error measured relative to the opening price. The other success criteria also failed.

04 · Decide

Rejected

All three R4 ideas failed their prewritten criteria. They did not change the final version.

Every offer is calculated. Every argument is sourced.
Every change must earn its place.

R4: offline experiments on 11 Oct; no arena sessions. The price-floor model, frozen-price policy and concession-curve model were rejected. Criteria and results ↗